USER MANUAL

DATA STORAGE, PUBLISHING AND VISUALISATION ENVIRONMENT

INDEX

INTRODUCTION AND SYSTEM ARCHITECTURE

This manual describes the complete workflow for maintaining, updating, and publishing data within the CTTC Geodata Explorer ecosystem.

The system design is based on two fundamental pillars: accessibility and scalability, allowing researchers, users, and technicians to manage the entire lifecycle of spatial data and update the web catalog by modifying a single structured configuration file.

MAIN COMPONENTS OF THE ENVIRONMENT

The system is centrally deployed on a Virtual Machine (VM) running Ubuntu and is divided into three functional layers.

1. **Storage (PostgreSQL/PostGIS)**: Dedicated exclusively to vector data, which is stored in a structured manner within spatial tables in a PostgreSQL database equipped with the PostGIS extension.

    - *Raster Data*: Due to specific performance requirements and limitations associated with direct native mosaic management in PostgreSQL, raster files are stored directly within a hierarchical file system on the VM disk, organized temporally using the `YYYYMMDD` naming convention.

2. **Publicación (GeoServer)**: The Apache Tomcat web application server hosts the GeoServer instance.
   GeoServer acts as the gateway that connects to the data sources, processes the information, and exposes it through interoperable OGC standards.

3. **Visualisation (Web Viewer**: An application built using JavaScript (OpenLayers), HTML, and CSS, packaged and optimized using the Vite development environment.
   The viewer dynamically consumes geographic web services published through GeoServer.

1. LOCAL DEVELOPMENT AND VIEWER MODIFICATION

To modify the viewer interface, add new functionalities, or update the layer catalog, it is necessary to run the frontend environment locally before deploying the changes to the Virtual Machine.

PREREQUISITES

- Node.js must be installed on the local machine (LTS version recommended).

INITIALISATION THE BASE PROJECT WITH VITE

From the local terminal or command line, run the following command:

```
None
npm create vite@latest
```

Durante el asistente interactivo de configuración, se deben seleccionar los siguientes parámetros:

- **Project Name:** *`project-name`* (Determines the name of the root development folder)
- **Framework:** *`Vanilla`* (Pure JavaScript, recommended for optimal native integration with OpenLayers)
- **Language**: *`JavaScript`*

INSTALLING GEOGRAPHIC DEPENDENCIES

Once the base structure has been created, access the project folder and install the main mapping libraries:

```
None
cd nombre-proyecto
npm install
npm install ol
npm install ol-layerswitcher
npm install proj4
```

- *`ol`*: OpenLayers core engine used for map rendering
- *`ol-layerswitcher`*: Control component for layer management and visibility toggling
- *`proj4`*: Coordinate transformation library used to support local coordinate systems (e.g., ED50 or ETRS89)

SOURCE CODE SYNCHRONISATION

The source code of the CTTC viewer application is hosted on GitHub.

- **Official Repository**: *https://github.com/Casas00/TFE-EGG-2026-DCP*

Clone / Replacement Instructions: Remove the files automatically generated by the Vite installation from your local project folder.

Download the repository as a ZIP archive and copy all files from the downloaded repository into the project folder (which should now be empty).

This preserves the Node module structure generated by Vite while replacing the default source code with the CTTC viewer implementation.

RUNNING THE LOCAL DEVELOPMENT VIEWER

To visualise code changes in real time, start the development server::

```
None
npm run dev
```

The application will be deployed locally, typically at: *http://localhost:5173*

2. INFRASTRUCTURE AND CONNECTION TO THE VIRTUAL MACHINE (VM)

All production server administration tasks require secure interaction with the Virtual Machine.

ACCESS CREDENTIALS

- **Remote Server (host)**: `geoexplorer.cttc.es`
- **System User**; `cttc`
- **Access Password**: `########`

REMOTE ACCESS VIA SSH

To open a secure terminal session within the VM, run the following command from your local machine:

```
None
ssh cttc@geoexplorer.cttc.es
```

Enter the password when prompted by the system.

FILE TRANSFER VIA SCP

The *scp* command allows transferring packaged files from the local environment to storage locations within the VM.

GENERAL SYNTAX

```
None
scp archivo.zip cttc@geoexplorer.cttc.es:/ruta/destino/
```

USE CASE 1 – UPLOAD THE VIEWER BUILD (DIST.ZIP)

```
None
scp dist.zip cttc@geoexplorer.cttc.es:/home/cttc/
```

USE CASE 2 – UPLOADING A RASTER DATASET PACKAGE (RASTER.ZIP)

```
None
scp raster.zip cttc@geoexplorer.cttc.es:/home/cttc/
```

3. ALMACENAMIENTO Y GESTIÓN DE DATOS VECTORIALES (POSTGRESQL/POSTGIS)

Vector geometric elements (points, lines, polygons, and their associated attribute tables) are managed centrally within the VM database.

BULK DATA LOADING WITH OGR2OGR

The import of common formats (Shapefiles, GeoJSON, GeoPackage) is performed directly from the VM terminal using the GDAL/OGR geospatial tool suite.

CREDENCIALES POSTGRESQL

- **Database name (dbname)**: `cttc_geomatics_ru`
- **User (user)**; `cttc_admin`
- **Password (password)**: `########`

IMPORT COMMAND STRUCTURE

```
None
ogr2ogr -f "PostgreSQL" \
      "PG:host=localhost       dbname=cttc_geomatics_ru       user=cttc_admin
      password=########" \
        /home/cttc/archivo.shp \
        -nln nombre_esquema.nombre_tabla \
        -nlt PROMOTE_TO_MULTI \
        -a_srs EPSG:25831 \
        -overwrite
```

The command is executed as a single line , The "\" symbol is used to indicate a line break

DESGLOSE DETALLADO DE PARÁMETROS CRÍTICOS

Comando	Explicación
-f "PostgreSQL"	Defines PostgreSQL as the destination driver format
"PG:host…"	Connection string containing the network parameters and credentials of the production database
/home/cttc/archivo.shp	Path to the geospatial file stored on the VM
-nln nombre_esquema.nombre_tabla	(***New Layer Name)*** Defines the destination

	schema and the name that will be assigned to the resulting physical table
`-nlt PROMOTE_TO_MULTI`	Forces geometry homogenization (e.g converts simple *Polygon* geometries into `MultiPolygon`), preventing constraint failures when merging records
`-a_srs EPSG:3857`	Assigns or explicitly declares the input Spatial Reference System in this case *WGS84 / Pseudo-Mercator*)
`-overwrite`	Completely overwrites and removes any preexisting table with the same name within the selected schema.

INTERNAL DATABASE ADMINISTRATION (POSTGRESQL CLI)

To perform maintenance tasks, table management and access control operations, it is necessary to connect the internal database manager.

1. Switch to the PostgreSQL service administrator user in Linux:

```
None
sudo -i -u postgres
```

2. Access the interactive console specifying the project database:

```
None
psql cttc_geomatics_ru
```

USEFUL INSPECTION COMMANDS IN THE PSQL CONSOLE

- **List available schemas:**

```
None
\dn
```

- **List tables within a specific schema:**

```
None
\dt nombre_esquema.*
```

- **List all tables in the database:**

```
None
\dt *.*
```

- **Create a new working schema (e.g., for a new project)**

```
None
CREATE SCHEMA nombre_esquema;
```

- **Grant full privileges on a schema to a user role:**

```
None
GRANT ALL PRIVILEGES ON SCHEMA nombre_esquema TO nombre_usuario;
```

4. RASTER FILE MANAGEMENT ON THE VM

Raster storage is natively located within the local file system of the Virtual Machine under the corporate root directory:

- **Raster Root Directory**: `/data/geoserver/rater/`

To verify the exiting raster datasets, execute:

```
None
sudo ls -l /data/geoserver/raster/
```

RECOMMENDED HIERARCHICAL STRUCTURE

To maintain layer organization and enable automated publishing, the creation of thematic subfolders by layer is required, together with standardized date-based file naming (*_AAAAMMDD.tif*)

```
None
/data/geoserver/raster/
└── [nombre_proyecto_o_dataset]/
    ├── [nombre_capa_1]/
    │   ├── nombre_capa_1_20260110.tif
    │   └── nombre_capa_1_20260115.tif
    └── [nombre_capa_2]/
        ├── nombre_capa_2_20260110.tif
        └── nombre_capa_2_20260115.tif
```

Real Example for the AirCrowd Project:

```
/data/geoserver/raster/aircrowd/
├── no2_mean/
│   ├── no2_mean_20190710.tif
│   ├── no2_mean_20190711.tif
│   └── indexer.properties
└── no2_surface/
    ├── no2_surface_20190710.tif
    └── timeregex.properties
```

To create a new thematic directory, use:

```
sudo mkdir -p /data/geoserver/raster/{nombre_directorio}
```

TEMPORAL MODULE CONFIGURATION (`ImageMosaic`)

When representing a time series composed of multiple successive raster images, GeoServer uses the *`ImageMosaic`* logical storage format.

This format indexes an entire directory f images and unifies them into a single raster layer.

To enable date extraction and temporal filtering, two mandatory property configuration files must be placed inside the layer directory together with the raster files.

1. *FILE: `indexer.properties`*

Defines the structural properties of the spatial index and the association with the time dimension

- **Creation / Editing:**
  `sudo nano /data/geoserver/raster/{dataset}/{capa}/indexer.properties`
- **Required Content:**

```
TimeAttribute=time
Schema=*the_geom:Polygon,location:String,time:java.util.Date
PropertyCollectors=TimestampFileNameExtractorSPI [timeregex](time)
```

2. *FILE:* `timeregex.properties`

Defines the regular expression used to extract the exact date by analysing the raster filename:

- **Creation / Editing:**
  `sudo nano /data/geoserver/raster/{dataset}/{capa}/timeregex.properties`

- **Required Content:**

```
None
regex=.*_(\d{8})\.tif
format=yyyyMMdd
```

This expression assumes that any raster file following the structure: `nombre_YYYYMMDD.tif`, contains an 8-digil numerical sequence corresponding to:

`YYYY` → Year
`MM` → Month
`DD` → Day

GeoServer automatically extracts this information and associates it with the internal temporal dimension of the `ImageMosaic` layer.

5. *CONFIGURATION AND PUBLISHING THROUGH THE GEOSERVER INTERFACE*

Once the data has been properly structured on the server (either in PostGIS or within raster directories), it can be published through the GeoServer graphical interface.

PUBLISHING VECTOR DATA (POSTGIS)

1. **Creating the Data Store:** From the left-hand menu, select **Data Stores** and click **Add New Store.** Choose ***PostGIS*** – ***PostGIS DataBase***

Almacenes de datos

Gestionar los almacenes que proveen datos a GeoServer

Agregar nuevo almacén Eliminar los almacenes seleccionados

Nuevo origen de datos

Seleccione el tipo de origen de datos que desea configurar

Origenes de datos vectoriales

- Directory of spatial files (shapefiles) - Takes a directory of shapefiles and expos
- GeoPackage - GeoPackage
- H2 - H2 Embedded Database
- H2 (JNDI) - H2 Embedded Database (JNDI)
- PostGIS - PostGIS Database
- PostGIS (JNDI) - PostGIS Database (JNDI)
- Properties - Allows access to Java Property files containing Feature information
- Shapefile - ESRI(tm) Shapefiles (*.shp)
- Web Feature Server (NG) - Provides access to the Features published a Web Fe

2. **Connection Parameters**: Define the target workspace and enter the environment parameters:

Parameter	Value
`Host`	`localhost`
`Port`	`5341`
`Database`	`cttc_geomatics_ru`
`Schema`	*(accroding to the project)*
`User`	`cttc_admin`
`Password`	`########`

3. **Publicación de la Capa**: Al guardar la conexión, GeoServer listará todas las tablas físicas del esquema. Seleccione la tabla deseada y pulse Publicar. Defina de forma obligatoria el **Nombre**, **Título**, verifique el **SRC Nativo** y **Declarado** (ej. `EPSG:3857` o `EPSG:4326`) y pulse las opciones de **Calcular desde datos** y **Calcular desde el encuadre nativo** para fijar el Bounding Box (Encuadres).

   After saving the connection, GeoServer will display all physical tables availables within the selected schema.

   Select the desired tables and click **Publish**

   The following parameters must be configured:

   - **Name**
   - **Title**
   - **Native CRS**
   - **Declared CRS**

   Examples:

```
None
    EPSG:3857
    EPSG:4326
```

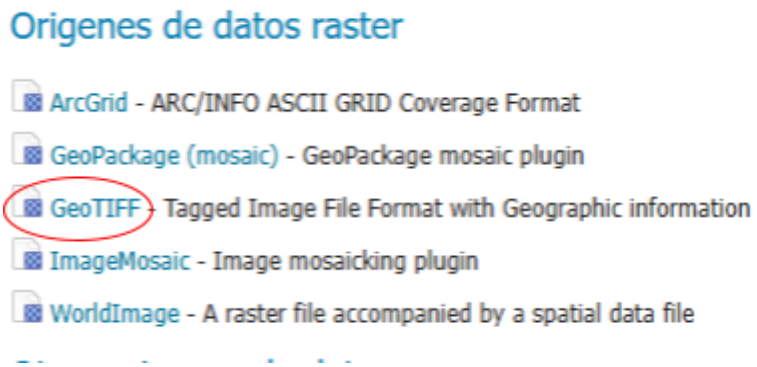
   To generate the layer extent, click **Compute from data** and **Compute from native bounds.** This will automatically populate the spatial Bounding Box.

4. **Activación Temporal (Time Dimension)**: If the table contains multitemporal information, it is necessary to configure the temporal dimension. Navigate to the **Dimension** tab and enable the **TIME** option.

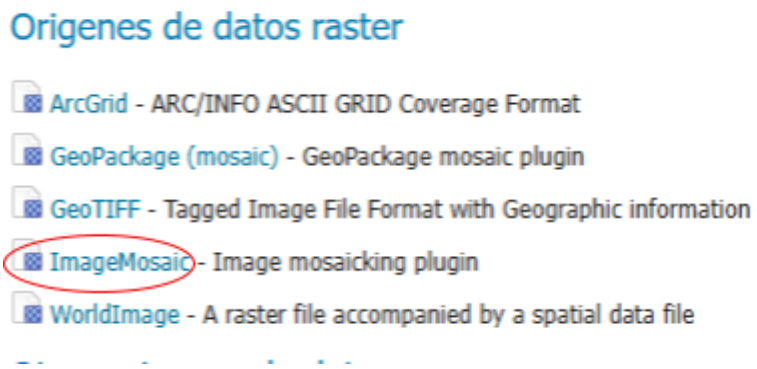
   GeoServer will automatically search for fields of type `Date` (with valid native data formats and typically name *date*) and associate them with the temporal dimension exposed through the WMS service.

PUBLISHING RASTER DATA (POSTGIS)

1. **Publishing Individual Raster Files:** Create a new data store of type **GeoTIFF.** This option is suitable for publishing a single raster file.



2. **Publishing Temporal Series (`ImageMosaic`)**: For temporal raster datasets composed of multiple files, select **ImageMosaic.**



3. **Mosaic Configuration**: Select the target workspace. Assign an identifier to the store and specify the URL pointing to the raster directory within the Virtual Machine using the formal file prefix::

   - `file:/data/geoserver/raster/{dataset1}/{raster1}/`

4. **Publishing and Time Dimension**: Guarde el almacén y haga clic en publicar capa. Complete los campos obligatorios del título y encuadres espaciales (Bounding Box). En la pestaña **Dimensions**, habilite **TIME**; si los archivos indexer.properties y timeregex.properties se configuraron correctamente en el paso anterior, GeoServer reconocerá la serie temporal del mosaico ráster sin problemas.

   Save the store and click **Publish Layer.**

   Complete the required parameters:
   - Layer Title
   - Bounding Boxes
   - Coordinate Reference System

   Then navigate to **Dimension** and enable **TIME.** If the `indexer.properties` and `timeregex.properties` files were correctly configured during the previous step, GeoServer will automatically recognize the temporal dimension of the raster mosaic.

ADDING DYNAMIC STYLES (SLD)

1. **Upload a Style**: From the **Styles** section in the left-hand panel, click **Add a New Style**.

   Define:
   - Style Name
   - Workscape
   - SLD/XML file

   Upload the SLD file developed locally and save the changes to register the style on the server.

2. **Linking the Style to a Layer**: Navigate to the configuration page of the published layer and select the **Publishing** tab.

   - **Default Style**: Assign the primary style that will be used when the layer is initially displayed in the viewer.

   - **Selected Styles**: Move any additional compatible styles to the list of selected styles. This configuration is critical because it allows the viewer to dynamically switch symbology based on user-selected attribute and visualisation options.

PERSONALIZACIÓN DE CONSULTAS INFORMATIVAS (FREEMARKER .FTL TEMPLATES)

By default, when a user clicks on the map to query a pixel or geometry (WMS *GetFeatureInfo* request), GeoServer returns a generic plain-text output.

To customize this output and integrate formatted HTML tables, the corresponding FreeMarker template (***content.ftl***) must be overridden within the physical workspace directory of the layer.

- **Template Storage Location on the VM**:
  /opt/tomcat/webapps/geoserver/data/workspaces/{workspace}/{datastore}/{layer_name}/content.ftl

- **Create / Edit the Template**:

```
None
sudo                                                                    nano /opt/tomcat/webapps/geoserver/data/workspaces/{workspace}/{datastore}/{layer_name}/content.ftl
```

- **Recommended and Adaptable Structure for *content.ftl***:

```
None
<style>
  /* Estilos generales de la tabla emergente */
  .gfi-table {
    border-collapse: collapse;
    width: 100%;
    max-width: 450px;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    font-size: 13px;
    color: #333;
    margin-bottom: 20px;
    border: 1px solid #ccc;
  }
  /* Estilo de los encabezados */
  .gfi-table th {
    background-color: #b4aed1;
    color: #333;
    font-weight: bold;
    padding: 8px;
    border: 1px solid #ccc;
    text-align: left;
    width: 40%;
```

```
  }
  /* Estilo de las celdas de datos */
  .gfi-table td {
    padding: 8px;
    border: 1px solid #eee;
    background-color: #fff;
  }
  .gfi-title {
    font-size: 14px;
    font-weight: bold;
    margin-bottom: 5px;
    color: #555;
  }
</style>

<#-- Inicio del bucle por cada elemento encontrado en el clic -->
<#list features as feature>
  <div class="gfi-title">Información de la Capa</div>
  <table class="gfi-table">

    <#-- BLOQUE DE EJEMPLO 1: Campo de texto estándar -->
    <tr>
      <th>Nombre del Atributo</th>
      <td>
        <#if feature.nombre_del_campo_en_bd.value??>
          ${feature.nombre_del_campo_en_bd.value}
        <#else>
          -
        </#if>
      </td>
    </tr>

    <#-- BLOQUE DE EJEMPLO 2: Campo numérico con formateo de decimales -->
    <tr>
      <th>Valor Numérico Certificado</th>
      <td>
        <#if feature.campo_numerico.value??>
          ${feature.campo_numerico.value?number?string("0.00")}
        <#else>
          -
        </#if>
      </td>
    </tr>

  </table>
</#list>
```

This template can be adapted to display any attribute available within the published layer, providing complete control over the appearance and formatting of information displayed in the viewer pop-up windows.

6. VIEWER CATALOG CONFIGURATION (`catalogConfig.js`)

One of the main advantages of this viewer is that it is not necessary to modify the core OpenLayers code in order to add, remove or update datasets.

The entire used interface, including layer menus, category structures, temporal controls, metadata panels, and styling options, is managed exclusively though the catalog configuration file.

- File Location: ***`src/map/layers/data/catalogConfig.js`***

The file exports an indexed constant called *catalogData*, structured as a JSON object hierarchy that automatically drives the behavior of the remaining modules within the application.

DETAILED DATASET STRUCTURE EXAMPLE

```JavaScript
export const catalogData = [
  {
    category: "Air Quality", // Main thematic category displayed in the viewer interface
    datasetInfo: { // General metadata associated with the dataset category
       description: "Air Quality dataset generated from satellite and in-situ observation for atmospheric analysis in Catalonia.",
      project: "AirCrowd",
      organisation: ["CTTC"],
      funding: "CTTC Research Initiatives",
      publications: [ // Scientific publications or related documentation
        {
          title: 'Air Quality Monitoring using Citizen Science',
          url: 'https://...'
        }
      ],
         logos: ['/logos/cttc.png'] // Logos displayed within the dataset information panel
    },
      subcategories: [ // Secondary groups displayed within the accordion structure
```

```
      {
        name: "NO2 - Satellite",
        layers: [ // Collection of layers associated with this subcategory
          {
             id: "no2surface", /// Unique internal identifier used by the viewer
logic
            name: "NO2 Surface", // Layer name displayed to the user
            workspace: "AirCrowd",// Exact GeoServer workspace name
            layerName: "no2_surface", // Published GeoServer layer name
            type: "wms", // OGC service type consumed by the viewer
            datatype: "raster", // Native dataset type ('raster' or 'vector'
              temporal: true, // Define si habilita el Slider temporal dinámico
en el visor
            time: { // Enables the temporal slider component
                 default: "latest", // Carga por defecto la fecha más reciente
disponible ('latest')
               format: "date", // Display format ('date' → YYYY-MM-DD, 'year' →
YYYY)
              available: [ // List of available timestamps used by the temporal
slider
                "2023-12-21", "2023-12-22", "2023-12-23", "2023-12-31"
              ]
            },
            styleConfig: { // OPTIONAL: Used when multiple styles are available
              type: "byYear", // Style transition behaviour
              variable: "attr1", // Variable portion of the style name
                    options: ["attr1", "attr2", "attr3"], // Available styles
published in GeoServer
                    pattern: "{year}_{variable}" // Naming convention used to
automatically generate WMS style requests
            },
                metadata: { // Information displayed when the user clicks the
"Info" button
              base: { // Mandatory metadata fields
                title: 'NO2 Surface',
                      description: 'NO2 concentration in the land surface in
Catalonia',
                source: 'CTTC - Geomatics Department',
                authorship: 'CTTC',
                dates: '2023 - 2024', // Temporal coverage of the dataset
                crs: 'EPSG:4326 - WGS84' // Coordinate Reference System
              },
              details: [ // Fully customizable key-value information fields
                { label: 'Geometry Type', value: 'Raster' },
                { label: 'Resolution', value: '10m' }
              ]
            }
          }
```

```
        ]
      }
    ]
  }
];
```

MAIN CONFIGURATION BLOCKS

The configuration file (***catalogConfig.js***) is organised into a hierarchical structure composed of three levels:

CATEGORIES

Represents the main thematic groups displayed in the catalog panel.

Example:

```
JavaScript
category: "Air Qulity"
```

SUBCATEGORIES

Used to organise layers within the thematic groups.

Example:

```
JavaScript
subcategories: [
       {
       name: "NO2 - Satellite"
       }
]
```

LAYERS

Represents the actual published datasets available for visualisation .

Each layer contains:

- Service connection information
- Temporal configuration
- Style configuration
- Metadata
- Visualisation behaviour

TEMPORAL CONFIGURATION

Datasets that support temporal navigation must define `temporal: true` and include a corresponding `time` block.

Example:

```javascript
time: {
       default: "latest",
       format: "date",
       available: [
              "2023-12-21",
              "2023-12-22",
              "2023-12-23"
       ]
}
```

PARAMETER DESCRIPTION

Parameter	Description
`default`	Initial timestamp loaded by the viewer (normally `"latest"`)
`format`	Display format used by the temporal slider
`available`	List of available timestamps

STYLE CONFIGURATION

The `styleConfig` block allows dynamic style switching without modifying the viewer source code.

Example:

```javascript
styleConfig: {
       type: "byYear",
       variable: "ndvi",
       options: ["ndvi", "ndwi", "evi"],
       pattern: "{year}_{variable}"
}
```

The viewer automatically generates style request using the configured pattern

METADATA CONFIGURATION

Metadata displayed through the information panel is defined inside the `metadata` block

Two levels are supported:

BASE METADATA

Standard information shared by all datasets:

- `Title`
- `Description`
- `Source`
- `Authorship`
- `Data Range`
- `Coordinate Reference System`

DETAILED METADATA

Custom key-value pairs specific to the dataset

Ejemplo:

```javascript
details: [
       { label: 'Resolution',
       value: '10m'
       }
]
```

This approach allows each dataset to expose its own technical characteristics without modifying the viewer source code.

7. PRODUCTION DEPLOYMENT (BUILD PROCESS AND TOMCAT SERVER)

Once the layers have been modified within the `catalogConfig.js` file or visual changes have been made to the local development code, the project must be compiled and deployed to the production Tomcat server.

STEP 1: GENERATING THE OPTIMISED BUILD

Open a command prompt (CMD) and navigate to the project directory where the viewer source code is stored: ***`cd visor-openlayers`***

Execute the following command:

```
None
npm run build
```

This command compiles, bundles, and minifies all HTML, CSS and JavaScript files, generating an optimised deployment directory called ***`/dist`***. The generated folder is ready for deployment to a web server environment.

STEP 2: PACKAGE AND UPLOADING

Compress the content of the ***`/dist`*** into a ZIP archive named ***`dist.zip`***. Upload the file to the root directory of the Virtual Machine using the ***`scp`*** command introduced previously:

```
None
scp dist.zip cttc@geoexplorer.cttc.es:/home/cttc/
```

STEP 3: DEPLOYMENT WORKFLOWS INSIDE THE SERVER (SSH CONNECTION)

Connect to the Virtual Machine via SSH and execute the following command in the specified order:

1. **Extract the Uploaded Package**

```
None
unzip /home/cttc/dist.zip
```

2. **Clean the Tomcat Root Directory (*`ROOT`*):**

Tomcat serves web applications from the directory ***`/opt/tomcat/webapps/ROOT`***.
To avoid conflicts, obsolete files, or cache inconsistencies with previous deployments, the existing directory should be completely removed an recreated.

```
None
sudo rm -rf /opt/tomcat/webapps/ROOT/
sudo mkdir /opt/tomcat/webapps/ROOT
```

3. **Copy the New Build Files:**

Copy the contents of the generated build into Tomcat's deployment directory

```
None
scp cp -r /home/cttc/dist/* /opt/tomcat/webapps/ROOT/
```

4. **Configure Ownership and Permissions**:

*For security reasons, the Linux user (**cttc**) and the user that executes the Apache Tomcat service (**tomcat**) belong to different system roles*

*If the files are copied without updating ownership, Tomcat may not have sufficient permissions to access them, resulting in HTTP errors such as **403 Forbidden,***

Transfer ownership recursively to the Tomcat service account and assign the recommended execution and read permissions:

```
None
sudo chown -R tomcat:tomcat /opt/tomcat/webapps/ROOT
sudo chmod -R 755 /opt/tomcat/webapps/ROOT
```

Permission explanation:

- 7 → Read + Write + Execute (Owner)
- 5 → Read + Execute (Group)
- 5 → Read + Execute (Others)

This configuration allows Tomcat to access and server all applications resources while maintaining a secure permission model

5. **Restart the Tomcat Service:**

To ensure that all application files are reloaded and caches resources are refreshed, restart the Tomcat service::

```
None
sudo systemcyl restart tomcat
```

6. **Verify Service Status:**

```
None
sudo systemcyl status tomcat
```

Expected result:

```
JavaScript
Active: active (running)
```

FINAL PRODUCTION VERIFICATION

Open a web browser and verify that the development has completed successfully by accessing the official viewer URL: :

- ***https://geoexplorer.cttc.es:8443/***

For advanced support regarding server administration, network infrastructure, GeoServer configuration, or spatial database troubleshooting, please contact the CTTC Geomatics Department.

This document provides the operational procedures required to maintain, update, publish, and deploy datasets within the CTTC GeoData Explorer environment.